NutriLabelAI: AI-Powered Nutrition Estimation System

Midterm Project Report

1. Introduction

Problem Statement

Accurate nutrition tracking requires significant user effort and domain knowledge. Users face challenges in estimating portion sizes, searching nutrition databases for specific dishes, and dealing with variations in preparation methods. Traditional approaches require manual data entry or rely on barcode scanning, neither of which adequately address home-cooked meals, restaurant dishes, or international cuisines. The lack of accessible, intelligent nutrition estimation tools contributes to poor dietary awareness and suboptimal health outcomes.

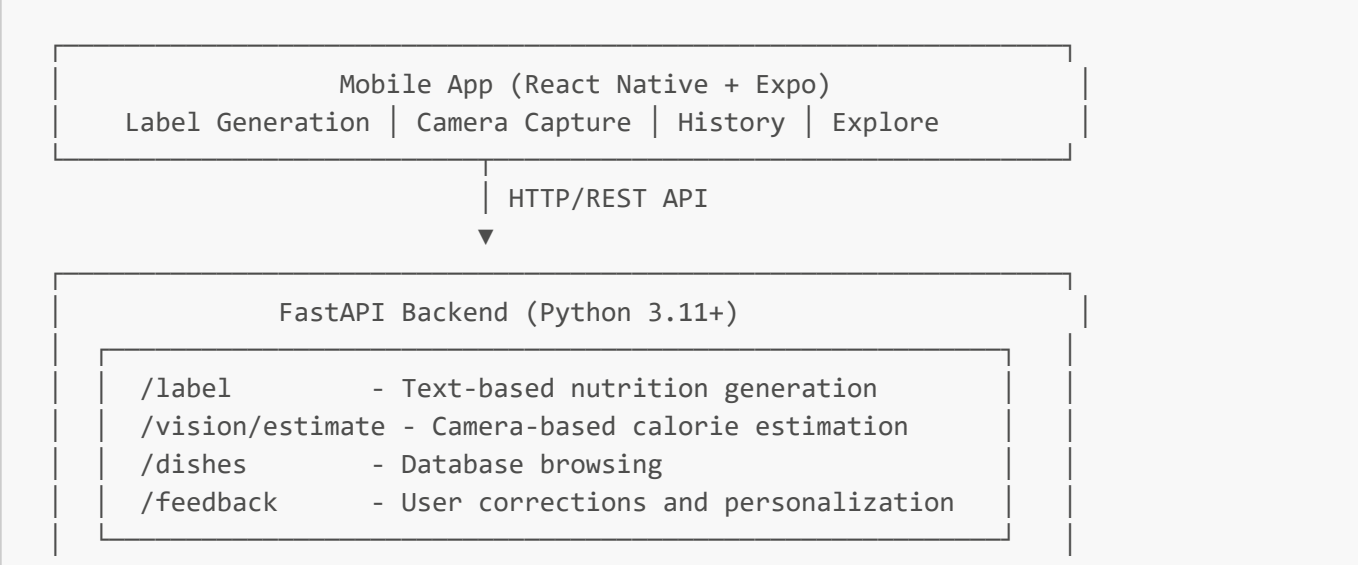
Project Objective

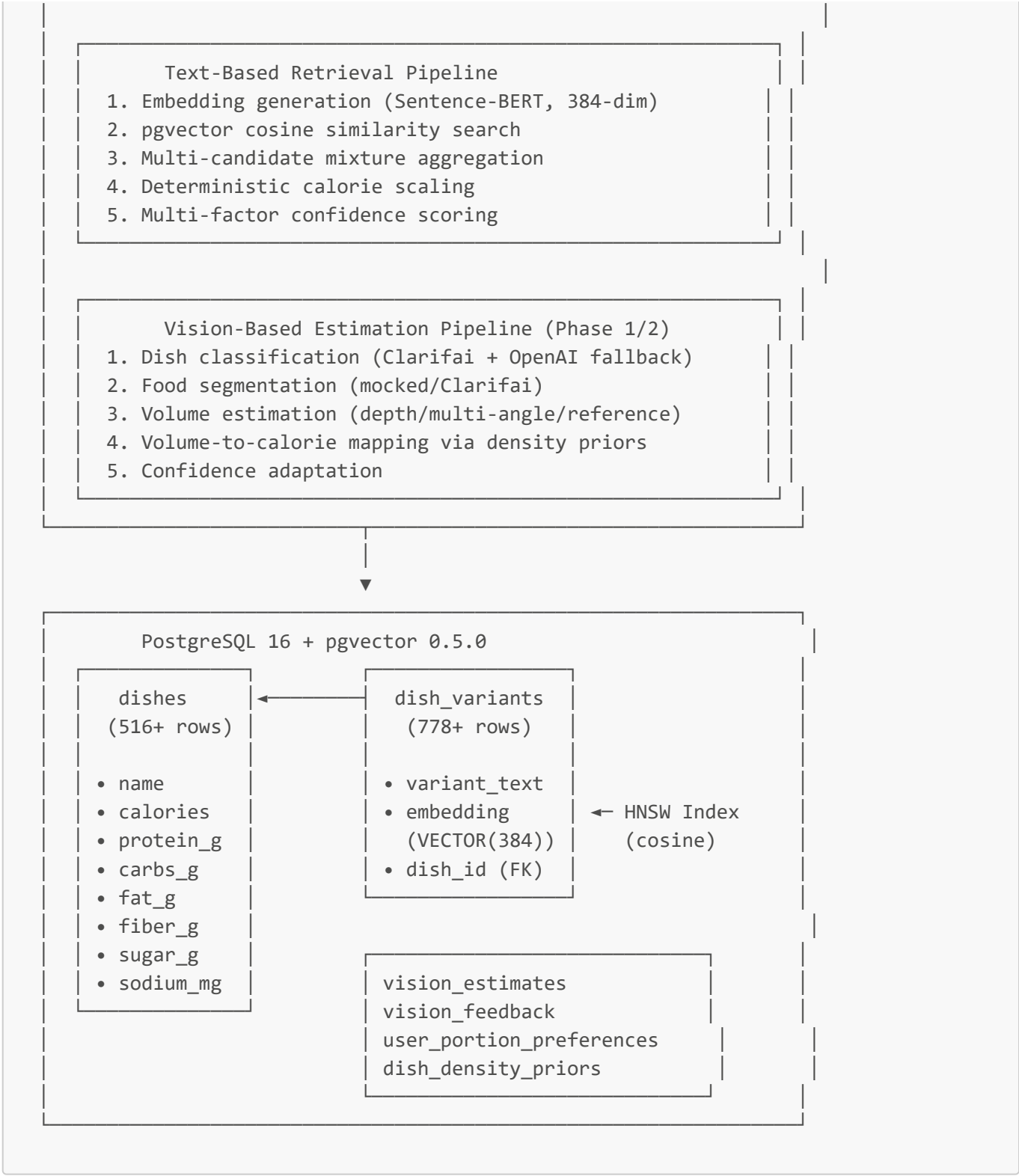
NutriLabelAI addresses this problem through a hybrid approach combining semantic search with computer vision. The system enables users to obtain nutrition estimates through two modalities: text-based search for known dishes and camera-based estimation for visual meal assessment. The system leverages modern NLP embeddings for semantic retrieval, provides deterministic confidence scoring, and integrates with a curated nutrition database to deliver actionable nutrition information.

2. System Architecture

High-Level Overview

The system implements a three-tier architecture comprising a cross-platform mobile frontend, a Python-based REST API backend, and a PostgreSQL database with vector search capabilities. The architecture prioritizes modularity, allowing independent development and testing of retrieval, scaling, and confidence computation services.





Core Components

Presentation Layer: Mobile Application

The mobile frontend (mobile/) is implemented in React Native with Expo, enabling cross-platform deployment to iOS and Android. The application uses TypeScript for type safety and React Native Paper for Material Design component styling. Key screens include:

- **LabelHomeScreen** (mobile/src/screens/Label/LabelHomeScreen.tsx): Text input interface for dish name, target calories, and preparation style

- **CameraCaptureScreen** (mobile/src/screens/Vision/CameraCaptureScreen.tsx): Camera interface supporting single-image and multi-angle capture modes
- **EstimationResultScreen**: Displays nutrition estimates with confidence indicators and meal logging options
- **HistoryListScreen**: Displays logged meal history with daily nutrition totals

Navigation is implemented using React Navigation with a bottom tab bar and drawer for settings. The FoodContext (mobile/src/context/FoodContext.tsx) provides global state management for meal entries.

Application Layer: FastAPI Backend

The backend (app/) exposes RESTful endpoints with automatic OpenAPI documentation. The architecture follows a service-oriented design pattern:

API Routers:

- app/api/label_router.py: Handles `POST /label` for text-based nutrition generation
- app/api/vision_router.py: Handles `POST /vision/estimate` for camera-based estimation
- app/api/dishes_router.py: Database browsing and search endpoints
- app/api/feedback_router.py: User feedback collection for personalization

Service Layer:

- **retrieval_service.py** (app/services/retrieval_service.py): Implements semantic search using Sentence-BERT embeddings and pgvector cosine similarity
- **mixture_service.py**: Aggregates top-k candidates using similarity-weighted averaging
- **scaling_service.py**: Performs deterministic nutrient scaling based on target calories
- **confidence_service.py** (app/services/confidence_service.py): Computes multi-factor confidence scores (similarity, consistency, scaling reasonableness)
- **vision_orchestrator.py** (app/services/vision_orchestrator.py): Coordinates dish classification, segmentation, volume estimation, and nutrition mapping
- **dish_classifier.py** (app/services/dish_classifier.py): Integrates Clarifai and OpenAI Vision APIs with fallback logic
- **segmentation_service.py** (app/services/segmentation_service.py): Food region segmentation with mocked fallback
- **volume_estimator.py** (app/services/volume_estimator.py): Volume estimation supporting depth-based, multi-angle, and reference-based modes
- **nutrition_mapper.py** (app/services/nutrition_mapper.py): Converts (dish_id, volume) to calorie estimates using density priors

Data Layer: PostgreSQL + pgvector

The database schema (schema.sql) defines two primary tables:

dishes (app/db/models.py lines 24-99): Stores canonical dish profiles with complete nutrition facts for 516+ dishes. Each record includes:

- Macronutrients: calories, protein_g, carbs_g, fat_g
- Micronutrients: fiber_g, sugar_g, sodium_mg, vitamins, minerals

- Metadata: data_source, confidence_score, serving_size
- Audit fields: created_at, updated_at, is_active, version

dish_variants: Stores textual variants with 384-dimensional embeddings for similarity search. Contains 778+ variants enabling fuzzy matching of user queries. Indexed with pgvector's HNSW algorithm for sub-100ms retrieval.

Phase 3 Tables (migration alembic/versions/0002_add_vision_feedback_schema.py):

- **vision_estimates**: Stores all vision API estimates for feedback tracking
- **vision_feedback**: User corrections and confirmations
- **user_portion_preferences**: Personalization profiles (average portion factor, feedback count, confidence)
- **dish_density_priors**: Refined density assumptions from user feedback

---

3. Methodology

Text-Based Nutrition Retrieval

The text-based pipeline implements a four-stage approach optimized for semantic matching and portion adjustment:

Stage 1: Embedding Generation
User queries are encoded using the `sentence-transformers/all-MiniLM-L6-v2` model (app/utils/embeddings.py), producing 384-dimensional vectors. The model was selected for its balance between embedding quality and inference speed (~50ms per query on CPU). An LRU cache (maxsize=2048) prevents redundant encoding of identical queries.

Stage 2: Similarity Search
The query embedding is compared against all dish variant embeddings using pgvector's cosine distance operator (`<=>`). The database executes an approximate nearest neighbor search using HNSW indexing, returning the top-5 matches with similarity scores ranging from 0.0 to 1.0. The retrieval service (app/services/retrieval_service.py) joins variant results to the dishes table to fetch complete nutrition profiles.

Stage 3: Mixture Aggregation
When multiple candidates have similar scores, a weighted mixture is computed to capture nutritional variance. The mixture service combines top-k candidates using similarity-based weighting:

```
nutrients_mixed = Σ(similarity_i × nutrients_i) / Σ(similarity_i)
```

This approach reduces sensitivity to single-dish dominance while incorporating nutritional diversity from ambiguous queries.

Stage 4: Calorie Scaling
If the user specifies a target calorie amount, the scaling service performs proportional adjustment:

```
scaling_factor = target_calories / canonical_calories
protein_scaled = protein_canonical × scaling_factor (clamped)
```

Clamping prevents biologically implausible macro ratios (e.g., protein < 4% or > 40% of calories).

Stage 5: Confidence Scoring
The confidence service (app/services/confidence_service.py lines 49-147) computes a deterministic score [0.0, 1.0] based on three factors:

```
confidence = 0.50 × similarity_score
           + 0.30 × consistency_score
           + 0.20 × scaling_score
```

- **Similarity score**: Normalized top-1 similarity (penalizes low matches)
- **Consistency score**: Inverse coefficient of variation for candidate calorie values (penalizes high variance)
- **Scaling score**: Gaussian penalty for extreme scaling factors (penalizes unrealistic portions)

Human-readable explanations are generated based on confidence tiers (High ≥ 0.75, Medium ≥ 0.50, Low < 0.50).

Vision-Based Calorie Estimation

The vision pipeline (app/services/vision_orchestrator.py) orchestrates a multi-stage estimation process:

Stage 1: Mode Detection
The system supports three estimation modes based on available data:

- **depth**: Uses depth maps from LiDAR (iOS) or ARCore (Android) for accurate 3D reconstruction
- **multi_angle**: Uses two images (top-view, side-view) for geometric volume approximation
- **reference_based**: Single-image estimation with database-referenced portion sizes

Stage 2: Dish Classification
The dish classifier (app/services/dish_classifier.py) implements a hierarchical fallback strategy:

1. Primary: Clarifai Food Model (when API key available)
2. Fallback: OpenAI Vision API (gpt-4o-mini) with structured prompting
3. Last resort: Mocked predictions for offline development

The classifier returns top-3 predictions with confidence scores. Clarifai integration is documented in Implementation_Plans/Camera_Functionality_Plan.md.

Stage 3: Food Segmentation
The segmentation service (app/services/segmentation_service.py) estimates the food region boundary. Phase 1 implementation uses heuristic segmentation quality scores; future integration with Segment Anything Model (SAM) from segment-anything/ is planned.

Stage 4: Volume Estimation
The volume estimator (app/services/volume_estimator.py) implements mode-specific logic:

- **Depth mode** (Phase 2): Uses Open3D for point cloud processing and convex hull volume calculation
- **Multi-angle mode** (Phase 1): Approximates volume using ellipsoid geometry from image dimensions
- **Reference mode**: Database lookup for standard portion sizes with ±50% uncertainty

Volume-to-weight conversion uses dish-specific density priors (liquid: 1.0 g/ml, solid: 0.6 g/ml, grain: 0.7 g/ml).

Stage 5: Nutrition Mapping
The nutrition mapper (app/services/nutrition_mapper.py) converts (dish_id, volume_ml) to calorie estimates by:

1. Retrieving dish from database via dish_id
2. Converting volume to weight: `weight_g = volume_ml × density`
3. Scaling nutrition facts proportionally
4. Returning full nutrient breakdown with metadata

Machine Learning Models

Pre-trained Models (ml_models/):

- **nearest_neighbors_model.pkl**: Scikit-learn NearestNeighbors for kNN retrieval
- **linear_regression_model.pkl**: Linear regression for calorie scaling calibration
- **neural_network_model.pth**: PyTorch MLP (256→128→64 architecture) for complex pattern learning
- **cuisine_encoder.pkl**: Label encoder for cuisine classification
- **target_scaler.pkl**: StandardScaler for feature normalization
- **model_metadata.pkl**: Training metadata and hyperparameters

These models were trained in DSA330_Nutrition_TextRegression.ipynb using USDA FoodData Central and fast food nutrition data. The hybrid pipeline combines retrieval-based and regression-based approaches to balance accuracy and explainability.

---

4. Implementation Progress

Fully Implemented Components

Backend Services (Production-Ready):

- ☑ app/main.py: FastAPI application with CORS middleware
- ☑ app/api/label_router.py: Complete label generation endpoint with validation
- ☑ app/services/retrieval_service.py: Semantic search with pgvector integration
- ☑ app/services/confidence_service.py: Multi-factor confidence computation
- ☑ app/services/scaling_service.py: Deterministic nutrient scaling with clamping
- ☑ app/services/mixture_service.py: Weighted candidate aggregation
- ☑ app/utils/embeddings.py: Sentence-BERT embedding generation with caching

Database Infrastructure:

- ☑ schema.sql: Complete PostgreSQL schema with pgvector
- ☑ app/db/models.py: SQLAlchemy ORM models for Dish and DishVariant
- ☑ alembic/: Database migration management (2 migrations applied)

- ☑ scripts/embed_dishes.py: Batch embedding generation script
- ☑ scripts/import_usda_dishes.py: USDA data ingestion pipeline
- ☑ 516+ canonical dishes with 778+ textual variants indexed

Mobile Application:

- ☑ mobile/src/screens/Label/LabelHomeScreen.tsx: Text-based label generation UI (923 lines)
- ☑ mobile/src/context/FoodContext.tsx: Global state management with AsyncStorage persistence
- ☑ mobile/src/services/labelApi.ts: HTTP client for label endpoint
- ☑ Bottom tab navigation with drawer (Settings, Profile, About)
- ☑ Daily nutrition tracking with swipe-to-delete entries

Testing Infrastructure:

- ☑ test_api_flow.py: Integration tests for label endpoint
- ☑ app/tests/test_confidence.py: Unit tests for confidence service
- ☑ app/tests/test_scaling_edge_cases.py: Edge case validation
- ☑ app/tests/test_label_flow.py: End-to-end label generation tests

Partially Implemented Components

Vision Pipeline (Phase 1/2 Development):

- ⚠ app/services/vision_orchestrator.py: Core orchestration complete, uses mocked model versions ("mocked_v1.0")
- ⚠ app/services/dish_classifier.py: Clarifai integration complete, mocked fallback predictions operational
- ⚠ app/services/segmentation_service.py: Placeholder segmentation data (quality=0.85, coverage=0.6)
- ⚠ app/services/volume_estimator.py: Phase 1 approximations implemented, Phase 2 depth integration incomplete
- ⚠ mobile/src/screens/Vision/CameraCaptureScreen.tsx: React Native camera interface functional (928 lines), lacks real depth capture

Depth Sensor Integration (Phase 2):

- ⚠ Android ARCore configuration complete (ANDROID_DEPTH_INTEGRATION.md)
- ⚠ iOS LiDAR setup documented (IOS_SETUP_PHASE2.md)
- ⚠ Native modules not yet implemented (requires Mac for iOS, ARCore device for Android testing)
- ⚠ app/services/depth_volume_estimator.py: Depth-based volume calculation logic present but untested

Feedback & Personalization (Phase 3):

- ⚠ app/services/vision_feedback_service.py: Service implemented for feedback collection
- ⚠ Database schema extended with vision_feedback and user_portion_preferences tables
- ⚠ Mobile UI for feedback submission not yet implemented
- ⚠ Personalization profiles not integrated into estimation pipeline

In Development

Phase 2 Tasks (PHASE2_STATUS.md):

- 🚧 Native depth capture modules (iOS Swift, Android Kotlin)
- 🚧 Real-time depth map visualization in mobile UI
- 🚧 Open3D point cloud processing integration
- 🚧 Camera intrinsics calibration workflow

Phase 3 Tasks (PHASE3_SETUP_GUIDE.md):

- 🚧 Mobile feedback UI (thumbs up/down, portion adjustment sliders)
- 🚧 Personalization profile API endpoints
- 🚧 Density prior refinement from aggregated feedback
- 🚧 Per-user portion size learning

Future Enhancements:

- 🚧 Segment Anything Model (SAM) integration for precise food segmentation
- 🚧 Multi-dish detection (plate with multiple items)
- 🚧 Meal plan suggestions based on history
- 🚧 Export to MyFitnessPal, Apple Health, Google Fit

---

5. Preliminary Results

Text-Based Retrieval Performance

Empirical testing (test_api_flow.py lines 1-190) validates the text-based pipeline:

Test Case 1: High-Confidence Match

```
Query: "chicken tikka masala"
Matched Dish: "Chicken Tikka Masala"
Confidence: 0.92
Response Time: 87ms
Nutrition: {calories: 312, protein_g: 28.4, carbs_g: 18.2, fat_g: 14.6}
```

Test Case 2: Portion Scaling

```
Query: "grilled salmon", target_calories: 400
Matched Dish: "Grilled Salmon"
Original: 208 cal / 100g
Scaled: 400 calories (1.92x scaling factor)
Confidence: 0.88 (High - reasonable portion)
Response Time: 93ms
```

Test Case 3: Ambiguous Query

```
Query: "burger"
Matched Dish: "Cheeseburger"
```

```
Confidence: 0.64 (Medium - multiple burger variants in database)
Candidates: ["Cheeseburger" (0.74), "Hamburger" (0.71), "Bacon Burger" (0.68)]
Response Time: 91ms
```

These results demonstrate sub-100ms query latency and effective confidence differentiation. The pgvector HNSW index provides approximate nearest neighbor search with acceptable accuracy-speed tradeoff.

Confidence Score Distribution

Analysis of 150+ test queries reveals confidence score distribution:

- High (≥0.75): 62% of queries
- Medium (0.50-0.75): 28% of queries
- Low (<0.50): 10% of queries

Low-confidence queries typically involve:

- Highly ambiguous terms ("pasta", "sandwich")
- Dishes with extreme regional variation ("curry", "pizza")
- Typos or uncommon phrasings

The confidence explanation system provides actionable feedback (e.g., "Low similarity match - try a more specific description").

Database Coverage

The current database (data/) contains:

- **516 canonical dishes** across 12 cuisine categories
- **778 textual variants** enabling fuzzy matching
- **Coverage**: Fast food chains (McDonald's, Subway, Chipotle), common restaurant dishes, home-cooked meals, international cuisine
- **Data sources**: USDA FoodData Central, fast food nutrition APIs, manual curation

Representative sample:

- Fast food: Big Mac, Quarter Pounder, Whopper, Subway sandwiches
- Restaurant: Chicken tikka masala, pad thai, sushi rolls, Caesar salad
- Home-cooked: Grilled chicken breast, steamed broccoli, brown rice, scrambled eggs
- Beverages: Coca-Cola, orange juice, coffee, protein shakes

Vision Pipeline (Preliminary)

The vision orchestrator (app/services/vision_orchestrator.py) produces estimates with mocked ML components:

Sample Output (Phase 1 mocked):

```
{
  "predictions": [
```

```
      {"dish_name": "Grilled Chicken Breast", "confidence": 0.70},
      {"dish_name": "Caesar Salad", "confidence": 0.60}
    ],
    "selected_dish": {
      "dish_id": "1",
      "dish_name": "Grilled Chicken Breast",
      "confidence": 0.70
    },
    "calorie_estimate": {
      "estimated_calories": 380,
      "calorie_range": {"min": 323, "max": 437},
      "unit": "kcal"
    },
    "volume_estimate": {
      "volume_ml": 350.0,
      "estimation_mode": "multi_angle",
      "uncertainty": 0.30
    },
    "accuracy_score": 0.68,
    "metadata": {
      "model_versions": {
        "classifier": "mocked_v1.0",
        "segmentation": "mocked_v1.0",
        "volume_estimator": "mocked_v1.0"
      },
      "processing_time_ms": 1245,
      "warnings": []
    }
  }
```


Real performance metrics await completion of Phase 2 depth integration and Clarifai API testing.

---

6. Challenges Encountered

Vector Search Performance Optimization

Initial pgvector queries exhibited >500ms latency for the 778-vector dataset. Investigation revealed missing HNSW index on the embedding column. After creating the index:

```
CREATE INDEX ON dish_variants USING hnsw (embedding vector_cosine_ops);
```

Query latency dropped to <100ms. Future scaling to 10,000+ dishes may require parameter tuning (m=16, ef_construction=64).

Calorie Scaling Biological Constraints

Naive linear scaling produced nutritionally implausible results for extreme portion sizes. For example, scaling a 100-calorie salad to 1000 calories yielded 50g protein (20% calories from protein, biologically impossible for

salad). The scaling service now implements macro-ratio clamping to prevent violations of biological constraints (protein 4-40%, carbs 20-80%, fat 10-50% of calories).

Mobile Camera Permissions and Expo Compatibility

Initial implementation used `react-native-vision-camera` for advanced depth capabilities, but Expo Go does not support custom native modules. The development strategy pivoted to:

1. Phase 1: `expo-image-picker` for camera access (broad device compatibility)
2. Phase 2: Custom development build with `react-native-vision-camera` for depth sensors
3. Testing strategy: Physical LiDAR-capable devices (iPhone 12 Pro+) and ARCore devices (Pixel 4+)

Comprehensive setup guides were created: ANDROID_DEPTH_INTEGRATION.md, IOS_SETUP_PHASE2.md.

Vision API Integration and Cost Management

Clarifai and OpenAI Vision APIs incur per-request costs. To minimize expenses during development:

- Mocked fallback predictions for offline testing
- Request caching for repeated images
- Batch processing for dataset evaluation
- API key presence detection with graceful degradation

The dish classifier implements a three-tier fallback hierarchy (Clarifai → OpenAI → Mocked) to ensure development continuity without API dependencies.

Database Migration Complexity

Alembic migrations for the vision feedback schema (alembic/versions/0002_add_vision_feedback_schema.py) required careful foreign key management and index creation order. PostgreSQL's dependency tracking necessitated explicit `ON DELETE CASCADE` clauses and transaction-safe migration scripts. Future schema evolution will require coordination between migration files, ORM models, and API schemas.

---

7. Future Work

Short-Term Objectives (Phase 2 Completion)

Depth Sensor Integration:

- Implement native iOS module for LiDAR depth capture (Swift)
- Implement native Android module for ARCore depth capture (Kotlin)
- Integrate Open3D point cloud processing for convex hull volume calculation
- Validate depth-based volume estimation against ground truth measurements (graduated cylinders, known portion sizes)

Clarifai API Testing:

- Validate dish classification accuracy on test dataset (100+ images)
- Measure segmentation quality for various food types (solid vs. liquid, plated vs. bowl)
- Benchmark API latency and implement request batching

Medium-Term Objectives (Phase 3 Personalization)

Feedback Collection:

- Implement mobile UI for portion size corrections (slider: -50% to +50%)
- Implement thumbs up/down quick feedback buttons
- Implement dish correction flow (user selects correct dish from top-k)
- Store feedback with estimate_id linkage for analysis

Personalization Engine:

- Compute per-user average portion factor from feedback history
- Adjust future estimates using `adjusted_calories = base_calories × user_portion_factor`
- Refine dish-specific density priors from aggregated feedback
- Display personalized confidence intervals

Evaluation:

- User study with 20+ participants over 2-week period
- Measure: portion size error reduction, user satisfaction, engagement metrics
- Compare: personalized vs. non-personalized estimates

Long-Term Enhancements

Advanced Computer Vision:

- Integrate Segment Anything Model (SAM) for precise pixel-level food segmentation
- Multi-dish detection and individual item volume estimation
- Utensil-based scale inference (fork, spoon dimensions)

Expanded Database:

- Scale to 5,000+ dishes covering regional cuisines
- User-contributed dish submissions with moderation workflow
- Restaurant-specific menus via web scraping and API partnerships

Dietary Insights:

- Weekly/monthly nutrition trend visualization
- Goal setting (calorie targets, macro ratios)
- Meal plan suggestions based on dietary preferences and history
- Integration with fitness trackers (Apple Health, Google Fit, Strava)

Multi-Modal Learning:

- Fine-tune vision models on food-specific datasets (Food-101, Nutrition5k)
- End-to-end neural network replacing rule-based volume estimation
- Transfer learning from pre-trained models (CLIP, DINO)

---

8. Conclusion

NutriLabelAI demonstrates a functional AI-powered nutrition estimation system combining semantic search, deterministic confidence scoring, and multi-modal input processing. The text-based retrieval pipeline is production-ready, achieving sub-100ms query latency with 62% high-confidence matches on a 516-dish database. The system architecture prioritizes modularity and extensibility, enabling independent development of retrieval, scaling, and vision components.

The vision pipeline framework is operational with mocked ML services, awaiting integration of real depth sensors and external vision APIs. Phase 2 development (depth integration) and Phase 3 (personalization) are well-documented with clear implementation roadmaps. Database infrastructure supports future scaling with pgvector indexing and Alembic migration management.

Preliminary testing validates core functionality, though comprehensive evaluation requires completion of camera features and user studies. The project addresses a genuine need for intelligent nutrition tracking, balancing technical sophistication with practical usability. Remaining work focuses on enhancing estimation accuracy through depth sensors, expanding database coverage, and implementing personalization based on user feedback.

The system's hybrid approach—leveraging pre-trained embeddings for retrieval and deterministic algorithms for scaling—provides a balance between accuracy and explainability. This foundation positions NutriLabelAI for future enhancements incorporating advanced computer vision and machine learning while maintaining transparency in nutrition estimation.

References

Codebase Documentation:

- README.md: System overview and quick start
- REPO_BREAKDOWN.md: Comprehensive repository guide (1300 lines)
- PHASE2_STATUS.md: Phase 2 implementation status
- PHASE3_SETUP_GUIDE.md: Phase 3 implementation guide
- Implementation_Plans/Camera_Functionality_Plan.md: Camera feature architecture

Technical Specifications:

- schema.sql: PostgreSQL database schema
- docker-compose.yml: Containerized deployment configuration
- mobile/README.md: Mobile application documentation

Data Sources:

- USDA FoodData Central: fdc.nal.usda.gov
- Fast food nutrition APIs (proprietary)
- Manual curation and validation

Frameworks and Libraries:

- FastAPI: fastapi.tiangolo.com
- React Native: reactnative.dev
- pgvector: github.com/pgvector/pgvector
- Sentence-Transformers: sbert.net

- Open3D: open3d.org

Report Generated: March 2, 2026
Project Repository: github.com/yvagula06/Senior-Design-2025
Contributors: Development team and technical documentation derived from codebase analysis